Wifi Call

LAN WebRTC Voice Communicator

Built by Atul Pandey | B.Tech CSE 2026 | March 2026

1. The Problem This Project Solves

Modern communication apps (WhatsApp, Discord, Google Meet) require an active, high-speed internet connection to route calls through centralized cloud servers. If the internet goes down, communication stops—even if the two people trying to talk are standing in the same room connected to the same WiFi router.

This project solves the "Local Dependency Problem" by completely removing the internet requirement. It enables two Android devices to discover each other automatically and establish a secure, full-duplex, high-quality audio call using strictly the Local Area Network (LAN).

2. When is this Useful?

- **Disaster Recovery & Emergency Scenarios:** When cell towers and ISPs fail during storms or earthquakes, devices can still communicate via a battery-powered local WiFi router or an Android Mobile Hotspot.
- **Secure Enterprise & Office Environments:** For internal office or warehouse intercoms where security policies prohibit routing company voice data through external third-party internet servers.
- **Remote Locations:** Worksites like deep-underground mines, offshore oil rigs, or remote campsites where a local network exists but outward internet access does not.
- **Zero-Latency Requirements:** Because the audio data travels directly from Phone A to Phone B without bouncing to a cloud server, the latency is practically zero, making it ideal for real-time local coordination.

3. Why is it Irreplaceable? (Unique Value Proposition)

Most "offline" walkie-talkie apps use raw UDP audio streaming, which suffers from massive packet loss, echo, and terrible audio quality. This project brings enterprise-grade WebRTC (the exact engine powering Google Meet and Zoom) to a purely offline environment.

- **Decentralized:** There is no central server. Every phone acts as its own server and client.
- **Self-Healing Audio:** WebRTC automatically handles acoustic echo cancellation, noise suppression, and variable bitrate encoding (using the Opus codec).
- **Air-Gapped Security:** The signaling and media streams never leave the physical building. It is mathematically impossible for an outside internet entity to intercept the call.

4. System Architecture

The application is built on a Clean Architecture pattern, divided into three distinct networking layers managed by a central State Manager.

A. The Discovery Layer (LanDiscoveryService)

Protocol: UDP (User Datagram Protocol) Broadcast.

Mechanism: Every 2 seconds, the app broadcasts a tiny JSON heartbeat to the subnet IP 255.255.255.255. Simultaneously, it listens for heartbeats from other devices.

Outcome: Maintains a dynamic, real-time list of available peers without needing a central registry server. Stale devices are pruned after 10 seconds of inactivity.

B. The Signaling Layer (SignalingService)

Protocol: TCP (Transmission Control Protocol) Sockets.

Mechanism: WebRTC requires devices to exchange complex connection data (SDP and ICE Candidates) before a call can start. The app spins up an internal TCP server on port 8889.

Robustness: Implements an Outgoing Queue to prevent Android socket exhaustion and utilizes Dart's LineSplitter to gracefully buffer and reassemble large TCP packets fragmented by local routers.

C. The Media Layer (WebRTCService)

Protocol: WebRTC (Unified Plan) / SRTP.

Mechanism: Handles the actual microphone capture and audio playback. It bypasses external STUN/TURN servers, forcing the WebRTC C++ engine to connect directly via local host candidates.

Hardware Integration: Intelligently delays speakerphone activation until after the network handshake completes to prevent native audio engine crashes on modern OS versions (Android 15 / HyperOS).

D. The State Manager (CallManager)

Acts as the central nervous system. It listens to the Signaling and Discovery layers, orchestrates the WebRTC handshake sequence (Offer → Answer → ICE Exchange), and exposes a simple ChangeNotifier state (idle, incoming, calling, connected) for the UI.

5. Tools & Technologies Used

• **Framework:** Flutter (v3.24+ compatible, utilizing the V2 Android Embedding)

• **Language:** Dart (Null-safe, asynchronous streams)

• **Networking Stack:** dart:io (Raw standard library sockets for maximum stability and precise TCP lifecycle control)

• **WebRTC Engine:** flutter_webrtc (v1.4.1+) — Binds Dart to the native Google WebRTC C++ library

• **Permissions Handling:** permission_handler — Manages Android runtime requests for Microphone and Local Network visibility

• **Identifier Generation:** uuid — Generates cryptographic unique IDs to prevent IP routing collisions on mobile hotspots

6. Engineering Challenges Overcome

1. **TCP Fragmentation Bug:** Standard routers chop large WebRTC SDP payloads into pieces. The app uses stream transformers to buffer network streams until a full newline is detected, preventing JSON decode crashes.

2. **ICE Candidate Race Conditions:** Local networks are so fast that connection data often arrives before the call is fully accepted. The app implements an in-memory queue to store and replay early data, preventing silent call failures.

3. **Android Socket Flooding:** WebRTC generates dozens of network paths instantly. The app queues outgoing TCP requests sequentially with a 100ms buffer, preventing native C++ engine aborts caused by opening too many file descriptors simultaneously.

7. Setup & Run Instructions

1. Clone the repository and run **flutter pub get**.

2. Connect two Android devices to the same WiFi router (or connect one device to the other's Mobile Hotspot).

3. Ensure both devices have accepted the Microphone and Local Network permissions.

4. Run **flutter run --release** on both devices.

5. Tap the green phone icon next to the discovered device to initiate the call.
